

The Testing Harness, Explained Two Ways

For every section below, the **Technical** part is the precise version, and the **Plain-English** part is how you'd explain it to someone non-technical. Read whichever you need.

A running example is used throughout: a scrubber that should wipe metadata from photos. We make 3 copies of the **same image** with different camera names in the metadata — **Canon**, **Nikon**, **Sony** — and watch what the scrubber does to them.

1. The problem this harness solves

Technical. A metadata scrubber has no test oracle: there is no canonical "correctly scrubbed file" to diff against, so we cannot test it by comparison to a known-good output. The harness sidesteps this with a **metamorphic relation**: construct inputs that differ **only** in metadata, scrub them, and assert the outputs no longer depend on that metadata. A surviving dependency is a leak. This is the test-oracle problem solved by relating outputs to each other instead of to a ground truth.

Plain-English. How do you check that a shredder really destroyed a document when you don't have a "perfectly shredded" copy to compare against? You can't. So instead you shred **three* documents that are identical except for the name on top*, and check whether you can still tell them apart afterward. If you can, the shredder failed. We never need a "correct answer" — we only need the outputs to become indistinguishable.

2. The one core idea (the whole engine)

Technical. Every check is the same statistical test — a between-group vs. within-group variance decomposition over categorical features. A **feature** is one observed location/quantity (e.g. the bytes where the camera name lived). For each feature we compute two counts:

- **max_within** — the largest number of distinct values the feature takes inside a single group (the group being "the same input scrubbed repeatedly").
- **n_between** — the number of distinct representative values the feature takes across the different groups.

The leak rule (**oracle/variance.py**):

```
leak = n_between > max(max_within, 1)
```

A feature leaks iff its variation **across** groups exceeds its variation **within** a group. The threshold is therefore never a hard-coded constant — it is the measured within-group noise.

Plain-English. One question, asked over and over: **"Does this thing change more between the different files than it randomly changes when I redo the exact same file?"** If a spot wobbles a lot even when nothing should be different, that wobble is just noise and we ignore it. If a spot is steady on reruns but differs across the three photos, *that* is a real signal — a leak. The clever part is we never guess how much wobble is "normal"; we measure it.

3. The noise floor (the measuring stick)

Technical. Before any leak test, take one representative input and scrub it **N** times (default 5). Any feature whose value differs across those N repeats is "variable" and forms the **noise floor** — the within-group axis of the variance decomposition (`oracle/floor.py`). For deterministic scrubbers (F1 / deterministic F2) the floor is provably empty, so any between-group difference is a leak by definition. For non-deterministic tools the floor is non-empty and the same rule applies against it. The floor is not a pass/fail check itself; it calibrates the threshold the other checks use.

Plain-English. First we learn the tool's natural "handwriting wobble." Sign your name 5 times — no two signatures are byte-for-byte identical, but they're all clearly *your* signature. We do the same to the scrubber: run it on one file five times and record what randomly changes (a timestamp, a random ID). That expected wobble becomes the bar. Only differences *bigger* than this natural wobble count as real.

4. Worked example: the rule in numbers

Technical. Observe the "camera-name" feature under three tools.

Tool	Same file ×5 → distinct values (<code>max_within</code>)	Across Canon/Nikon/Sony (<code>n_between</code>)	<code>n_between > max(max_within,1)</code>	Verdict
Leaky (keeps name)	1 (<code>Canon</code> , <code>Canon</code> ,...)	3 (<code>Canon</code> , <code>Nikon</code> , <code>Sony</code>)	<code>3 > 1</code>	LEAK
Clean (wipes all)	1 (empty each time)	1 (empty, empty, empty)	<code>1 > 1</code> false	no leak
Clean but noisy (random timestamp)	5 (5 timestamps)	3 (random)	<code>3 > 5</code> false	no leak

The third row is the key result: a genuinely clean tool whose randomness is loud does **not** trigger a false positive, because the between-group variation does not exceed the measured floor.

Plain-English.

- The **leaky** tool: redo the same photo and the camera name is always the same, but across the three photos it's Canon/Nikon/Sony. Steady on reruns, different across files → that's a leak. Caught.
- The **clean** tool: blank every time, in every direction → nothing to see. Pass.
- The **clean-but-noisy** tool: it stamps a random time, so even reruns of one photo look different. The across-photo difference doesn't stand out against all that random noise → we correctly say "no leak." No false alarm. *This is why we measure the wobble first.*

5. Four checks, one engine — just change "what a group is"

Technical. The decomposition is fixed; only the grouping factor changes.

Grouping factor	Within-group =	Between-group signal =	Role / module
repeat (same input ×N)	— (*defines* the floor)	n/a	yardstick — <code>oracle/floor.py</code>
metadata variant (same content, different metadata)	repeat noise	metadata leak	A1 — <code>oracle/leak.py</code>
source (same producer, peer files)	repeat noise	source fingerprint	A2 — <code>oracle/peerset.py</code>
input (everything varies)	n/a	a feature invariant across *all* inputs = tool signature	fingerprint guard — <code>oracle/fingerprint_guard.py</code>

The fingerprint guard inverts the question: instead of variation *across* groups it looks for features *constant across every input*, then subtracts the format's declared `mandatory_constants()` (magic numbers, required structure). The remainder is the scrubber's own signature.

Plain-English. Same machine, four jobs — the only thing you change is *what you hold the same and what you let vary*:

1. **Floor** — same file over and over → "how much does this tool wobble on its own?" (the measuring stick).
2. **A1 — metadata leak** — same picture, different metadata → "can you still tell which metadata went in?"

3. **A2 — source fingerprint** — different cameras, same picture → "can you still tell *which camera* took it?" (e.g. all Camera-X files share a quirk Camera-Y's don't).
 4. **Fingerprint guard** — let *everything* vary and look for what **never** changes → that's the tool stamping *itself* on every file (a tell-tale like a factory watermark). It reveals *which tool you used*.
-

6. How an A1 test actually runs, step by step

Technical. (`runner/run.py` → `oracle/leak.py`)

1. **Build corpus** (`corpus/synthetic.py`): write N copies each of 3 variants sharing **byte-identical content**; variant *i* carries one sentinel record (`AAAA...` / `BBBB...` / `CCCC...`). Content-identity is guaranteed by construction (the TOYF codec writes the same content bytes), so a failure implicates the scrubber, not the corpus.
2. **Measure the floor** (`oracle/floor.py`): scrub variant 0 N times.
3. **Scrub the full grid**: every variant × every repeat.
4. **Diff into features** (`oracle/diff.py`): align outputs and emit features like `byte@40:16` — a 16-byte run at offset 40 where outputs disagree.
5. **Decompose** (`oracle/variance.py`): apply the leak rule per feature. Any leaking feature → `Leak(kind="variant_correlated")` with a byte locus + hex context; cell verdict = FAIL.

Plain-English. Make three near-identical files (same picture, different metadata marker — think highlighter colors AAAA / BBBB / CCCC). Learn the tool's wobble by redoing one file a few times. Run the tool on all of them. Lay the results side by side and circle every spot where they disagree. For each circled spot, ask the one question from §2. If a spot still tracks the highlighter color, the metadata leaked — and we can point at the exact bytes.

7. The three test-dummy scrubbers (proof the alarms work)

Technical. Three fixture scrubbers over the synthetic TOYF corpus form the exit gate (`stubs/`):

- **CleanStub** — strips all metadata → deterministic, A1 **PASS**.
- **LeakyStub** — keeps the first record → variant-correlated A1 **FAIL** (the sentinel survives in its value bytes).
- **VersionStub** — appends `TOYSCRUB/1.2.3` to every output → A1 **PASS** (constant across variants ⇒ not variant-correlated) but fingerprint guard **FAIL**. This demonstrates that the guard catches a class of leak A1 structurally cannot.

Plain-English. Before trusting a smoke detector, you test it with real smoke. These three fake tools are that smoke:

- **Clean tool** — does its job → alarm stays quiet (good).
 - **Leaky tool** — deliberately leaves one piece of metadata → alarm fires (good).
 - **Sneaky tool** — wipes metadata properly *but* signs every file `TOYSCRUB/1.2.3`. The metadata-leak alarm correctly stays quiet (the signature is the same on all files, so it's not *metadata* leaking). A *different* alarm — the one that spots the tool's own watermark — fires instead. That's exactly why that second alarm has to exist.
-

8. The output: the Pareto matrix

Technical. Each (format, tool) produces one JSON document (`runner/matrix.py`) validated against `schemas/pareto_matrix.schema.json`. Cells are keyed by (adversary, fidelity) over {A1, A2} × {F1, F2, F3}, each carrying a verdict (`pass` / `fail` / `not_applicable` / `not_tested`), the measured noise floor, a list of leaks with evidence pointers, and (F3 only) a perceptual block. A tool-global `scrubber_fingerprint` block sits outside the cells. A3 cells are emitted for shape parity but marked `not_tested` / `out_of_scope_threat_model` — no A3 check is ever built.

Plain-English. The result is a small scorecard per tool: a grid of pass/fail boxes for each kind of attacker (A1, A2) at three processing intensities (F1/F2/F3), plus the "did it sign its own work?" result, plus receipts showing exactly which bytes leaked. One attacker type (A3) is listed but greyed out as "out of scope, not tested," on purpose.

9. Two contracts kept separate

Technical. `contract.py` defines two protocols with different lifecycles:

- **`Scrubber`** — the thing under test, a black box: `run(in, out, fidelity)`. May be any binary in any language, wrapped by `SubprocessScrubber`.
- **`FormatPlugin`** — harness-side format knowledge: `matches()` (magic dispatch), `annotate(offset)` (name a structure), `canonical_content()`, `mandatory_constants()`.

Separating them lets the harness test a scrubber it knows nothing about while still gaining format-specific localization when a plugin exists.

Plain-English. Two roles are kept apart: the **tool being tested** (we treat it as a sealed box — we don't care how it works inside) and the **harness's own knowledge of the file format** (used only to make better, more readable reports). Keeping them separate means the harness can test *any* tool, even one we know nothing about.

One-line takeaway

One statistical test — `"does it vary more between files than within reruns?"` — applied four ways by changing what you hold constant, proven with three deliberately-broken test tools, reported as a small pass/fail scorecard.